University of Sunderland

Faculty: BSC(Hons) Computer System Engineering

NAME: Rahul Chaudhary

Student ID: 250818187

MODULE CODE: CET138

MODULE TITLE: Full Stack Development

MODULE LEADER: David Grey

SUBMISSION DATE AND TIME: Fri, 19 Jun 2026

ASSIGNMENT: 1

Academic Misconduct is an offence under university regulations, and this involves:

- **Plagiarism** – where you use information from another information source (including your previously submitted work) and pass it off as your own. This can be through direct copying, poor paraphrasing and/or absence of citations.
- **Collusion** – where you work too closely, intentionally, or unintentionally, with others to produce work that is similar in nature. This can be through loaning of materials, drafts or through unauthorised use of a fellow student's work.
- **Asking another person to write your assignment** – where you ask another individual or company to complete your work for you, be that paid or unpaid, and submit it as if it were your own.
- **Unauthorised use of artificial intelligence** – where you use artificial intelligence tools to generate your assignment instead of completing it yourself and/or where you have not been given permission to use artificial intelligence tools by your module leader. Please complete the following declaration around you use of artificial intelligence tools in your assignment.

STATEMENT ON USE OF ARTIFICIAL INTELLIGENCE TOOLS:

• I have used artificial intelligence tools to generate an idea for my assignment:	**YES**
• I have used artificial intelligence tools to write my assignment for me:	**NO**
• I have used artificial intelligence tools to brainstorm ideas for my assignment:	**NO**
• I have used artificial intelligence tools to correct my original assignment:	**NO**

DECLARATION

- I understand that by submitting this piece of work I am declaring it to be my own work and in compliance with the university regulations on Academic Integrity.
- I confirm that I have done this work myself without external support or inappropriate use of resources.
- I understand that I am only permitted to use artificial intelligence tools in line with guidance provided by my Module Leader, and I have not used artificial intelligence tools outside this remit.
- I confirm that this piece of work has not been submitted for any other assignment at this or another institution prior to this point in time.
- I can confirm that all sources of information, including quotations, have been acknowledged by citing the source in the text, along with producing a full list of the sources used at the end of the assignment.
- I understand that academic misconduct is an offence and can result in formal disciplinary proceedings.
- I understand that by submitting this assignment, I declare myself fit to be able to complete the assignment and I accept the outcome of the assessment as valid and appropriate.

CET138 Full Stack Development

Student Name: Rahul Chaudhary

Student Id: 250818187

Email: kit25c.rhl@ismt.edu.np

Module Leader: David Grey

Module Code: 138

Module Title: Full Stack Development

Live demo Link of JavaScript:

https://rahulchaudhary12-debug.github.io/Demos/demo-js.html

JavaScript

What us JavaScript

The behavior layer of the Web is JavaScript — it makes pages interactive and dynamic. The CodeCraft project has two areas in which it uses java script: the main.js file is used to drive the entire codecraft.html single page application (SPA) (i.e., navigation, animation, counters, form handling etc.) and demo-js.html is a standalone calculator to show basic JavaScript concepts in isolation.

Live Demo – JavaScript Calculator

Link: https://rahulchaudhary12-debug.github.io/Demos/demo-js.html

See the calculator in action: Live Demo: CodeCraft JavaScript Calculator

This calculator is 100% JavaScript (no libraries), has a dark background screen, a 4x5 button grid, four arithmetic keys, AC, +/- and % Utility keys, floating-point correction, division by zero error handling and supports full keyboard control.

Calculator – core JavaScript Concepts

State Management

```
// Three variables hold the complete calculator state
let current  = '0';    // the number currently shown on screen
let previous = null;   // the first operand (stored after operator pressed)
let operator = null;   // the pending operation: '+', '-', '*', '/'
let justCalc = false;  // flag: did we just press = ?

const screen = document.getElementById('screen');
const expr   = document.getElementById('expr');
```

DOM Manipulation and Rendering

```
function render(val) {
  screen.className = 'display-main';
  let display = String(val);
  // Truncate long decimals to 8 significant figures
  if (display.length > 10) {
    const num = parseFloat(val);
    display = isNaN(num) ? 'Error' : num.toPrecision(8).replace(/\.?0+$/, '');
  }
  screen.textContent = display;  // update the DOM
}
```

Event Handling – Buttons and Keyboard

```
// Button clicks use inline onclick attributes in the HTML
<button class="btn btn-num" onclick="press(7)">7</button>
```

```
<button class="btn btn-op"  onclick="setOp('+')">+</button>
<button class="btn btn-eq"  onclick="calculate()">=</button>

// Keyboard support — listens on the whole document
document.addEventListener('keydown', function(e) {
  if (e.key >= '0' && e.key <= '9') press(parseInt(e.key));
  if (e.key === '+') setOp('+');
  if (e.key === '-') setOp('-');
  if (e.key === '*') setOp('*');
  if (e.key === '/') { e.preventDefault(); setOp('/'); }
  if (e.key === 'Enter' || e.key === '=') calculate();
  if (e.key === 'Escape') clearAll();
});
```

Arithmetic Logic and Error Handling

```
function calculate(chain) {
  if (operator === null || previous === null) return;
  const a = previous;
  const b = parseFloat(current);
  let result;
  if (operator === '+') result = a + b;
  if (operator === '-') result = a - b;
  if (operator === '*') result = a * b;
  if (operator === '/') {
    if (b === 0) { showError('Cannot ÷ 0'); return; }  // guard
    result = a / b;
  }
  // Fix floating-point noise: 0.1 + 0.2 = 0.30000...04
  result = parseFloat(result.toPrecision(12));
  current = String(result);
  render(result);
}

function showError(msg) {
  screen.className = 'display-main error';
  screen.textContent = msg;
  setTimeout(clearAll, 1600);  // auto-clear after 1.6s
}
```

Main.js – JavaScript Powering CodeCraft

SPA Page Navigation

```
function showPage(id) {
  // Hide all pages
  document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
  // Show the requested page
  document.getElementById('page-' + id).classList.add('active');
  // Update active nav link
  document.querySelectorAll('.nav-links a').forEach(a => a.classList.remove('active'));
  const link = document.getElementById('link-' + id);
  if (link) link.classList.add('active');
  window.scrollTo(0, 0);
}
```

Animated Counters (data-target attribute)

```
document.querySelectorAll('.stat-num').forEach(el => {
  const target = parseInt(el.dataset.target);  // reads data-target="10000"
  let count = 0;
  const step = Math.ceil(target / 80);
  const timer = setInterval(() => {
    count += step;
    if (count >= target) { count = target; clearInterval(timer); }
    el.firstChild.textContent = count.toLocaleString();
  }, 20);
});
```

Summary

JavaScript brings CodeCraft to life — from the SPA navigation system to the animated counters, from the contact form toast to the full-featured calculator. The key JavaScript skills demonstrated are:

- **DOM Manipulation** – Accessing and updating webpage elements using methods such as querySelector(), getElementById(), textContent, and class List.
- **Event Handling** – Responding to user actions through addEventListener() and button click events.
- **Data and State Management** – Storing and updating calculator values using variables like current, previous, and operator.

- **Decision Making and Logic** – Using conditional statements (if, else, and switch) to control program behavior.
- **Data Type Conversion** – Converting and formatting values with functions such as parseFloat(), parseInt(), String(), and toPrecision().
- **Timers and Animation Effects** – Creating dynamic counters and automatic message handling with setInterval() and setTimeout().
- **Custom Data Attributes** – Accessing HTML data-* attributes through the Dataset API to build animated counter functionality.